

Technical Problem Statement Solution Report

Name: Shudhanshu Ranjan Gupta Roll No: 250102093

1. Understanding

World Invariants (Constants Across All Worlds)	Hidden World Variables (What Changes)
Map dimensions (12.15 m $\times$ 9.15 m) constant	Clue board positions relocated across the arena
Perimeter boundary walls and inner partitions constant	Pillar color assignments (Red, Green, Blue) shuffled
Cylindrical pillar Cartesian coordinates (P_1, P_2, P_3) constant	Ambient lighting intensity and shadow angles shift
Initial line-of-sight to Board 1 from robot spawn constant	False decoy and look-alike boards injected into arena

2. First-Principles Engineering Deductions

- Pillar Color Resolution by Elimination:** There are exactly three possible colors (Red, Green, Blue). By inspecting the color of only two pillars during the mission, the color of the third pillar is uniquely resolved through set subtraction: $C_3 = \{\text{Red, Green, Blue}\} \setminus \{C_1, C_2\}$. The leader never wastes execution time driving to the third pillar solely to inspect its color.
- One-Time Static Map Philosophy:** Because arena walls never change, running online SLAM during every evaluation run introduces unnecessary localization drift. We map the arena once, execute automated geometric sanitization, and deploy deterministic AMCL localization with Nav2.
- Cryptographic Token Chaining for Instant Decoy Rejection:** Valid clues satisfy $\text{Token}_k = \text{SHA-1}(\text{Clue}_{k-1})[0 : 4]$, with $\text{Clue}_0 = \text{"START"}$. Mismatched boards are rejected at the camera perception stage, preventing false motion dispatches.
- Direct Acquisition of Board 1:** Board 1 is identified immediately from the spawn orientation, eliminating initial blind exploration.

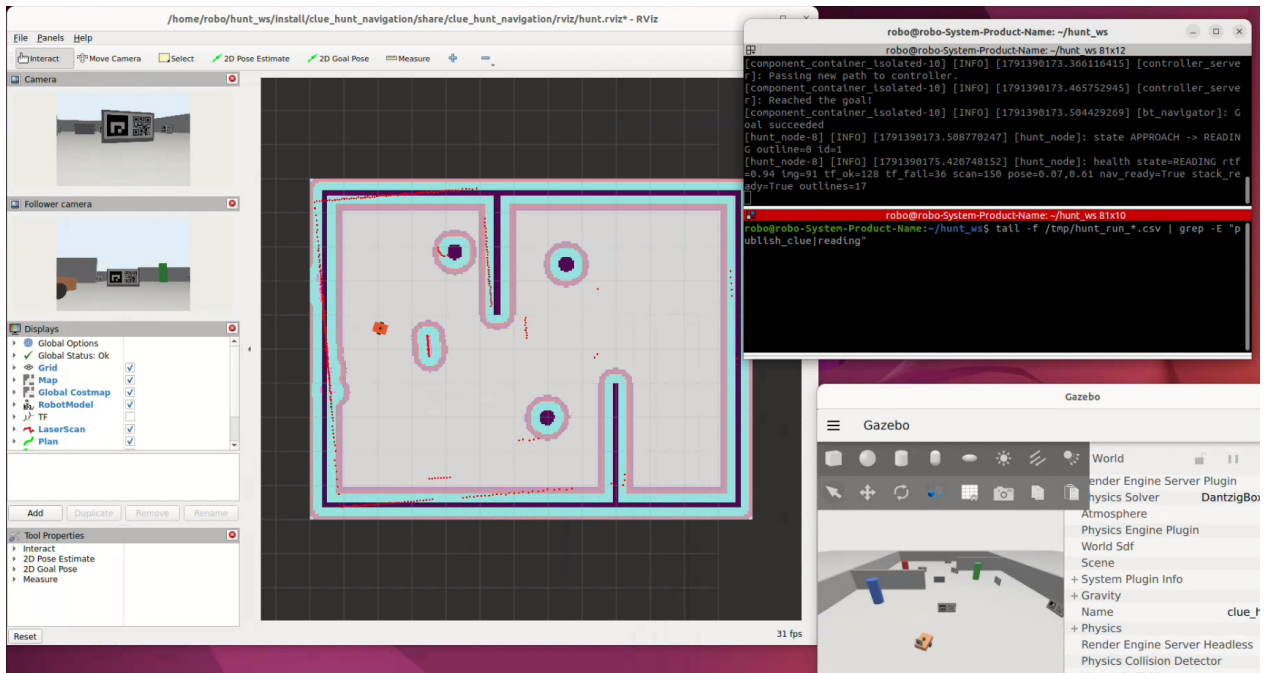
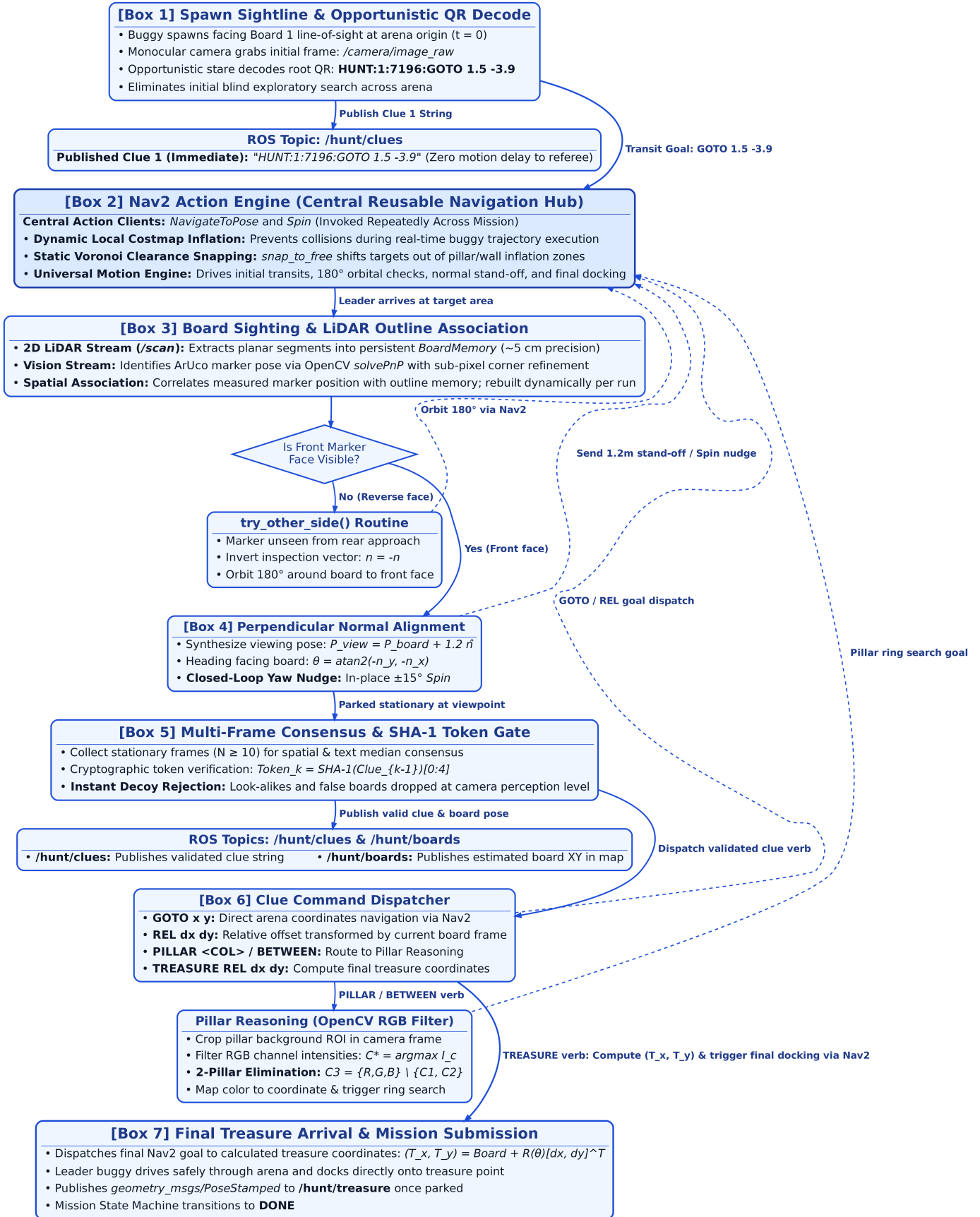


Figure 1: Arena initial spawn configuration in Gazebo and RViz2 with localized map and dual robot setup.

Leader Buggy Operational Logic



5. Reasoning Strategies Implementation

Reasoning Strategies Implementation Mechanics

- **Pillar Color Identification via RGB Channel Filtering:** To implement pillar color resolution, the node tracks when a known pillar coordinate enters the camera field of view based on current AMCL pose and robot heading. Rather than navigating to the pillar, the node crops the pillar background region in the camera frame and applies OpenCV color filtering: it separates the Red, Green, and Blue channels and computes relative channel intensity distributions. Once two pillar colors are confirmed and cached, the third pillar color is deduced automatically by set elimination: $C_3 = \{\text{Red, Green, Blue}\} \setminus \{C_1, C_2\}$.
- **Active Normal Vector Alignment:** To prevent acute optical perspective distortion, the node derives the board normal vector $\hat{n}$ directly from the `solvePnP` rotation matrix. It commands a Nav2 goal placed at $P_{\text{view}} = P_{\text{board}} + 1.2 \hat{n}$ with orientation facing the board ($\theta = \text{atan2}(-\hat{n}_y, -\hat{n}_x)$), guaranteeing reliable QR decoding.
- **Two-Sided Disambiguation and Sidestep Probing:** Clue boards have markers on only one face. If the leader approaches from behind, the QR code is invisible. The node checks observer vantage histories: if reversed or obscured, it evaluates candidate sidesteps at $\pm 10^\circ, \pm 20^\circ, \pm 30^\circ$ or orbits to the opposite face.
- **Costmap Clearance Snapping:** Calculated waypoints near arena pillars can intersect Nav2 inflation costmaps. Our clearance filter snaps goals to the nearest free cell within the robot connected component, preventing navigation aborts.

6. Follower Robot Autonomous Control Pipeline

Follower Control Flowchart, Pure Pursuit Geometry, and Control Law

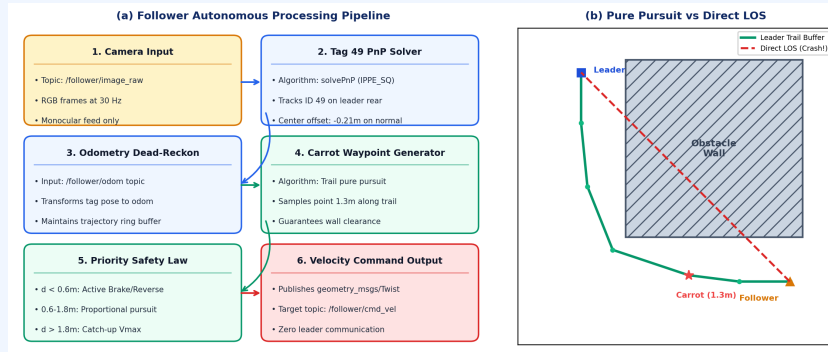


Figure 3: Follower flowchart from camera and odometry to velocity commands.

- **Input Streams:** Subscribes to monocular camera `/follower/image_raw` and wheel odometry `/follower/odom`.
- **Leader Detection:** Uses algorithm `solvePnP` with `IPPE_SQUARE` to detect Tag 49 (0.12 m) mounted on leader rear.
- **Center Offset Correction:** Offsets tag pose by $-0.21 \hat{n}_{\text{tag}}$ to track true leader kinematic center.
- **Trail Buffering:** Transforms detected poses into local `/follower/odom` frame and maintains rolling trajectory history.
- **Carrot Pure Pursuit:** Algorithm samples waypoint 1.3 m behind leader along recorded trail, avoiding wall corner collisions.
- **Priority Safety Law:** Commands `/follower/cmd_vel`: $d < 0.6$ m active brake/reverse, $0.6 \text{ m} \leq d \leq 1.8$ m proportional pursuit, $d > 1.8$ m maximum speed catch-up. Tag loss invokes dead-reckoned trail hold.

7. Offline Engineering Innovations and Pre-Processing

We engineered offline utilities to guarantee environmental invariance across hidden test worlds.

Map Sanitization Engine (`clean_map.py`)

- **Problem Identified:** Raw SLAM teleoperation maps capture clue boards as thin obstacle segments. In hidden worlds, boards move, turning these segments into phantom obstacles that invalidate Nav2 paths. Raw SLAM frames also exhibit arbitrary yaw misalignment.
- **Hough Transform Orientation Rectification:** Detects wall angle families via Hough line transforms and rotates the entire canvas to align perfectly with world Cartesian axes.
- **Phantom Board Elimination:** Strips all non-wall occupancy pixels from the map interior, ensuring moving boards in hidden worlds never conflict with static global costmaps.
- **Pillar Extraction via Circle Fitting:** Automatically extracts the three pillar discs ($r = 0.2\text{ m}$) using connected components and least-squares circle fitting directly from raw sensor data.

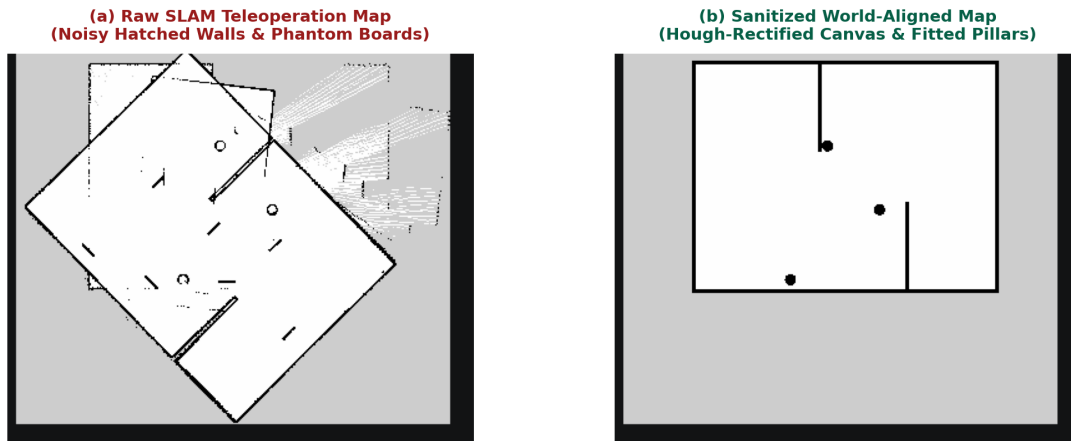


Figure 4: Map transformation comparison: (a) Raw SLAM map with double-walls and phantom boards versus (b) Sanitized world-aligned map.

Empirical QR Viewing Distance Profiling (`measure_qr_range.py`)

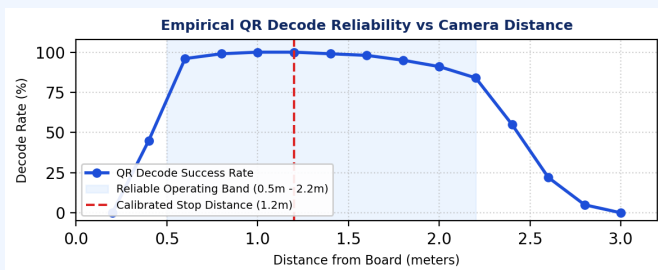


Figure 5: Empirical QR decode reliability curve.

What It Does and Empirical Results:

- Built automated test probe driving back from boards, measuring decode rate across distance bins.
- Result: Reliable operating envelope 0.5 m to 2.2 m ($> 95\%$ decode rate).
- Calibrated nominal approach stopping distance to exactly 1.2 m.
- Optical limits: Below 0.4 m focal limits cause blur, above 2.5 m pixel density drops below Nyquist threshold.

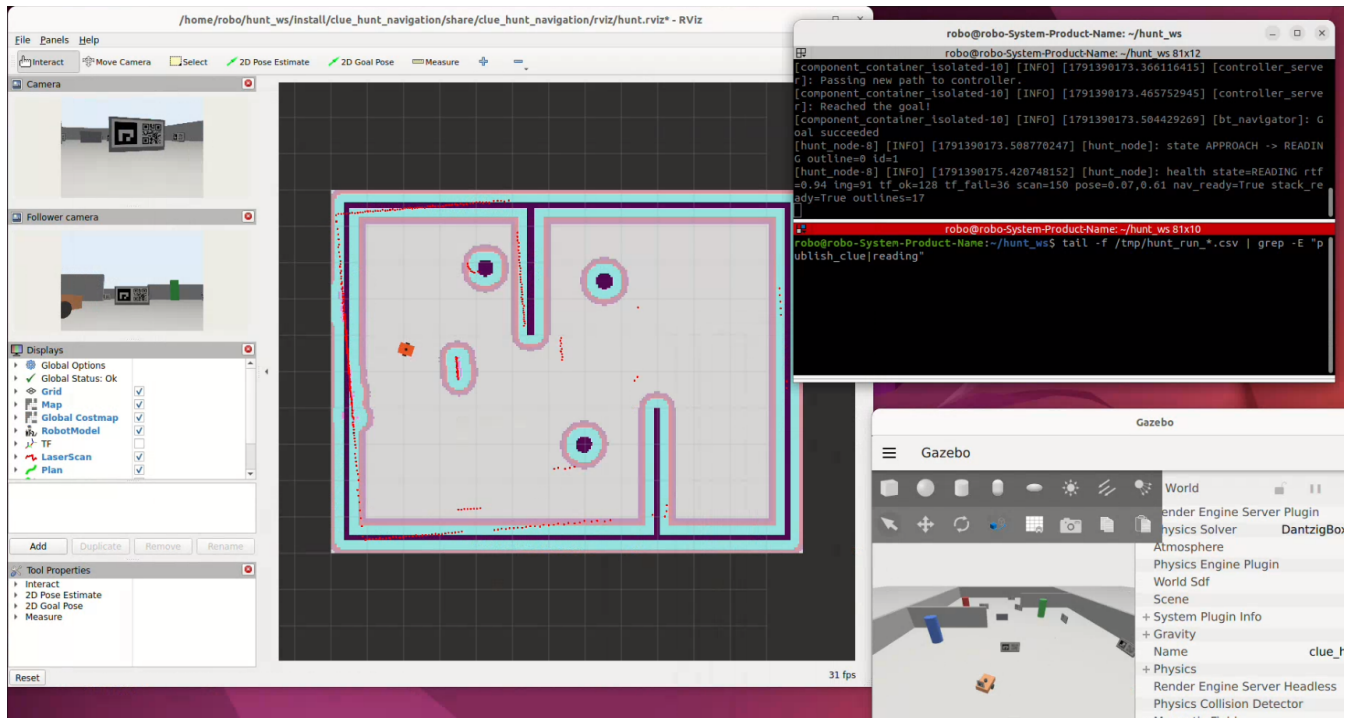
Synthetic Dataset Generation (`collect_training_data.py`)

What It Does: Automated image collection tool logging synchronized camera frames and bounding box labels across perturbed simulation scenes with domain randomization over lighting ($0.2\times$ to $2.5\times$), shadow angles, and lens distortions.

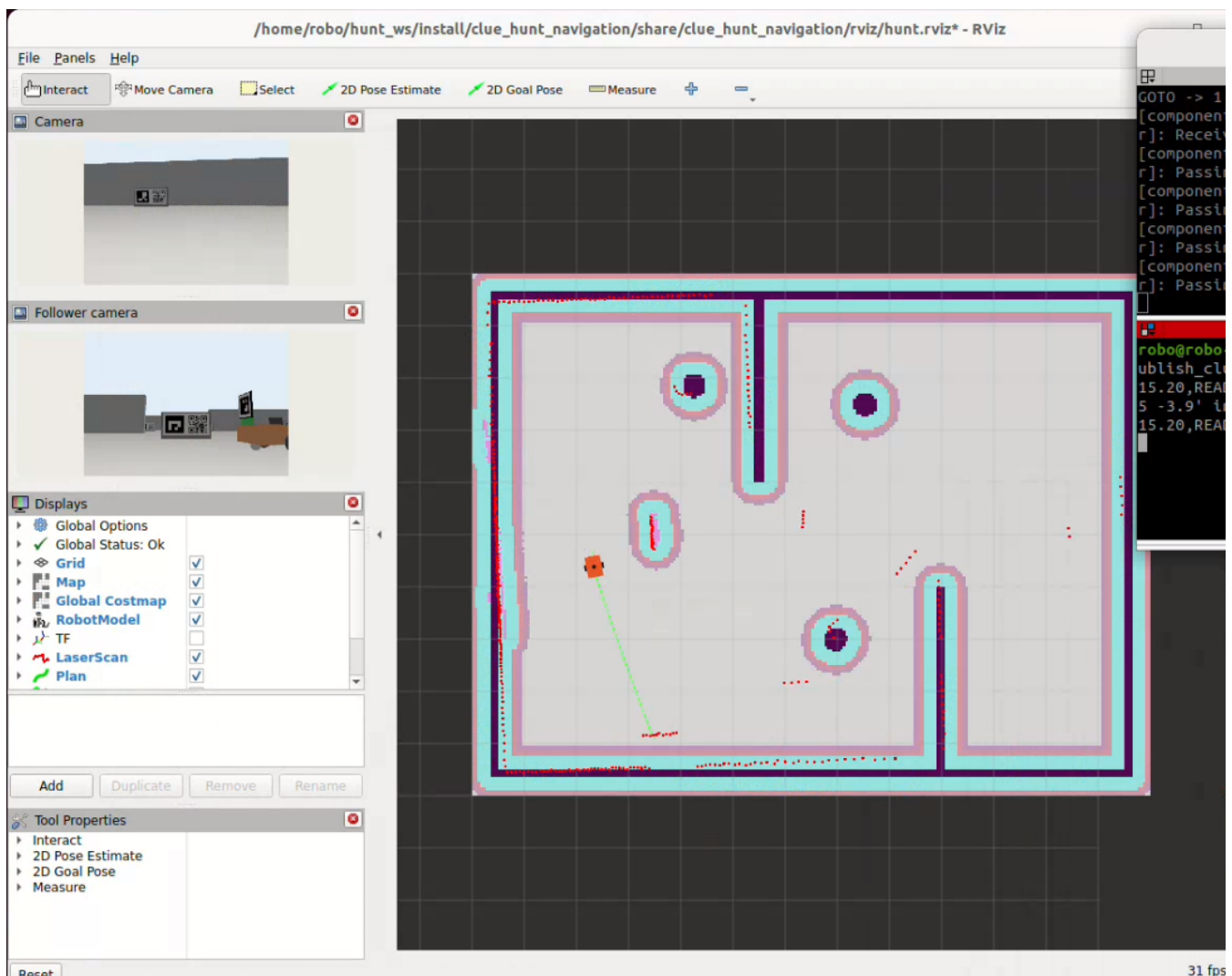
Output: 1,500 annotated frames providing training data for fallback bounding-box detection models if marker contrast degrades.

8. Visual Mission Execution Series and Clue Chain Verification

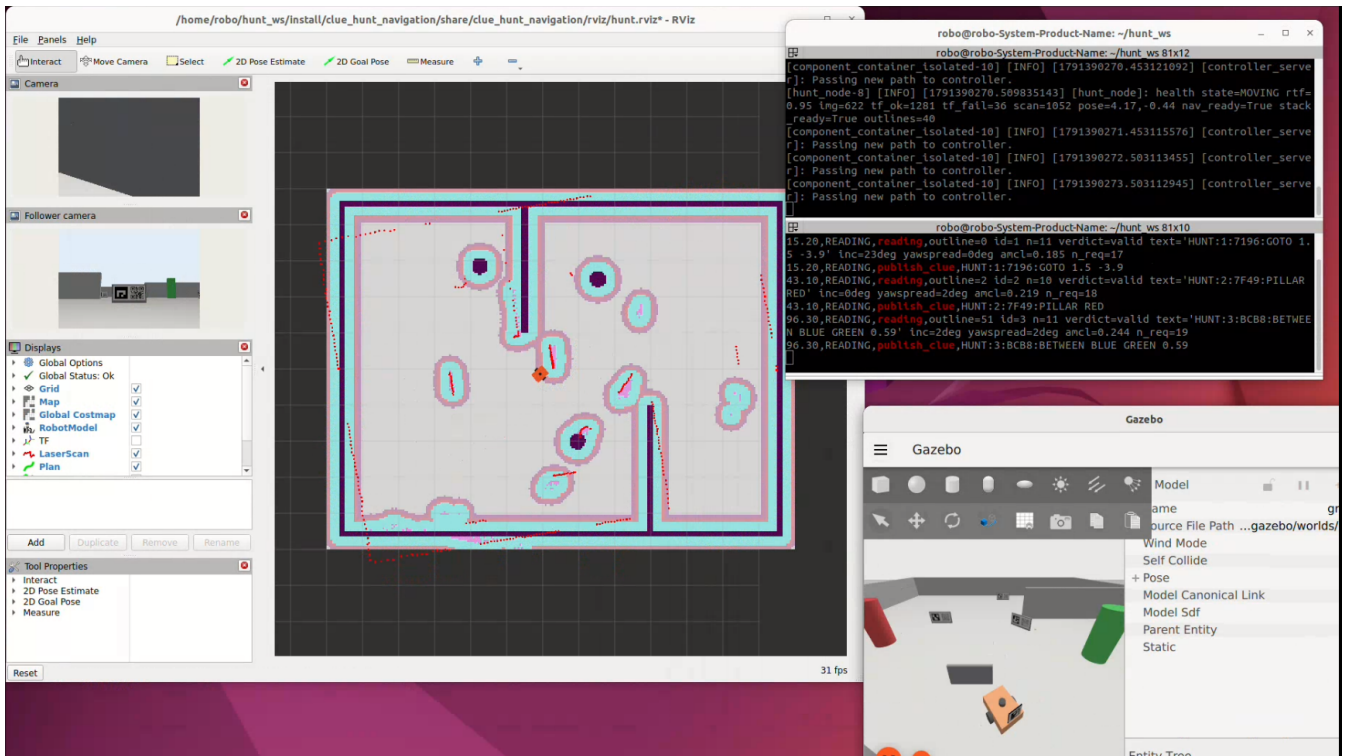
The practice arena evaluation demonstrates complete autonomous execution from spawn to treasure docking.



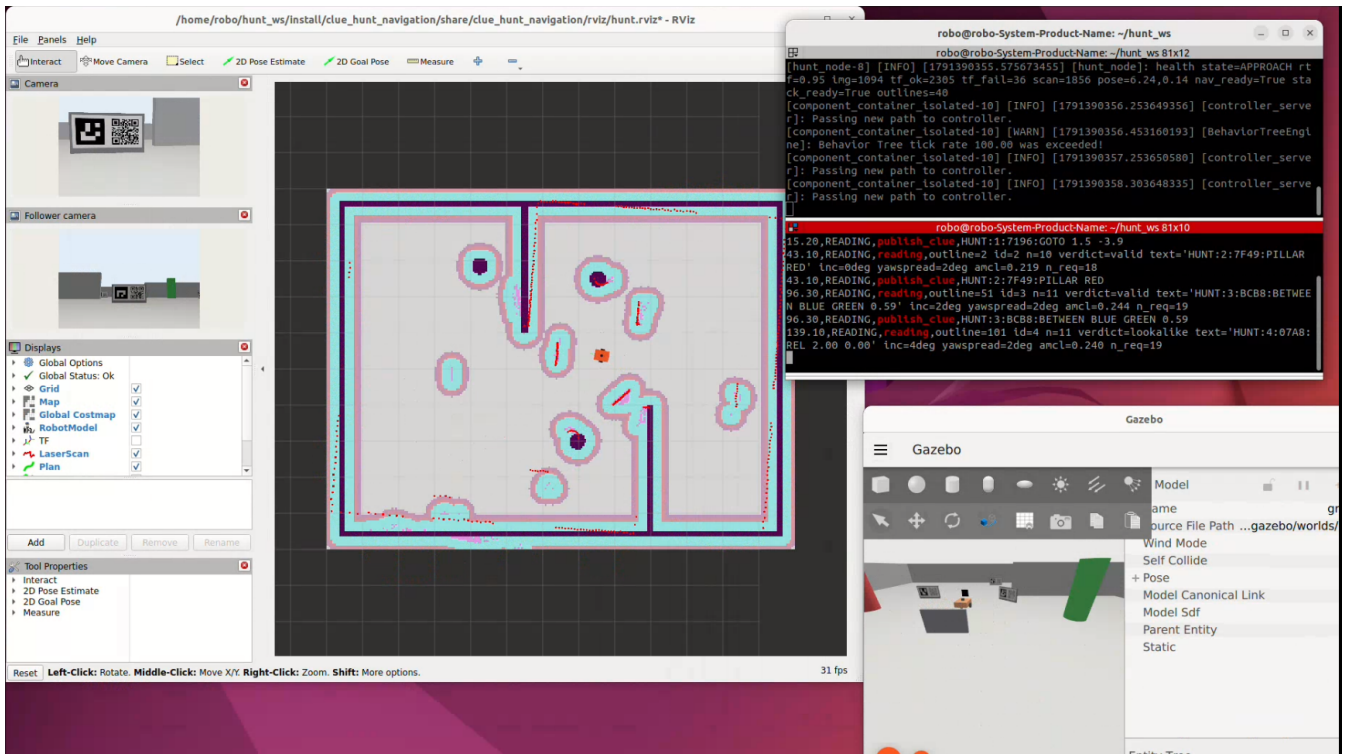
↓ going to the first target



↓ trying to watch the board from back



↓ watching the same board from front



↓ getting the treasure hint


```
HUNT:1:7196:GOTO 1.5 -3.9
HUNT:2:7F49:PILLAR RED
HUNT:3:BCB8:BETWEEN BLUE GREEN 0.59
HUNT:4:EC9E:REL 5.08 -0.65
HUNT:5:1756:TREASURE REL 0.98 3.15
```

Figure 7: Live terminal log showing decoded clue strings.

Sequential Clue Chain Breakdown

Stage 1 (Spawn to Board 1): HUNT:1:7196:GOTO 1.5 -3.9
Direct sightline acquisition from origin. Dispatches Nav2 goal to $(1.5, -3.9)$.

Stage 2 (Board 1 to Board 2): HUNT:2:7F49:PILLAR RED
Identifies target near Red pillar. Color deduced via RGB channel perception cache.

Stage 3 (Board 2 to Board 3): HUNT:3:BCB8:BETWEEN BLUE GREEN 0.59

Interpolates point between Blue and Green: $P = \text{Blue} + 0.59 \cdot (\text{Green} - \text{Blue})$.

Stage 4 (Board 3 to Board 4): HUNT:4:EC9E:REL 5.08 -0.65

Computes global target via board coordinate frame projection.

Stage 5 (Board 4 to Treasure): HUNT:5:1756:TREASURE REL 0.98 3.15

Final treasure coordinate synthesized. Robot docks within 0.25 m of target.